

trams-agent V2 架构设计文档

本文定义 trams-agent 下一阶段的目标架构。V2 包含三项核心架构特征：**前后端分离 (Vue SPA)**、**Embedding 与向量检索进程内化**、**面向低延迟的流水线设计**。本文聚焦架构体系，不涉及具体代码实现细节。

目录

- 1. [设计目标与约束](#)
- 2. [架构总览](#)
- 3. [部署架构](#)
- 4. [通信架构](#)
- 5. [数据流与流水线](#)
- 6. [RAG 子系统架构](#)
- 7. [延迟预算与低延迟设计](#)
- 8. [管理控制台 \(配置中心\)](#)
- 9. [安全与可观测性](#)

一、设计目标与约束

1.1 核心目标

目标	指标
端到端延迟	1 秒以内 (段就绪 → 译文推出)
数据零外传	音频/文本/译文全部留在私网
术语一致性	RAG 术语库约束翻译
前端可独立演进	Vue SPA 独立构建部署
模型可控	prompt / 模型 / 术语库自主掌握

1.2 关键约束

- **目标硬件**: NVIDIA DGX Spark 单机, 单 GPU (约 64GB 显存, 多容器共享)
- **零中间件**: 无 Redis、无消息队列、无独立向量库服务
- **模型容器外挂**: 大模型用 vLLM 容器化, 业务后端通过 OpenAI 兼容协议调用

1.3 延迟口径定义（重要）

本文档的"1 秒以内"指标，口径明确界定为 "段就绪 → 译文推出"：

- 指标口径**：从"一段音频被 VAD 切好"到"译文推送到前端"，必须 $\leq 1s$ 。这是 V2 流水线可控、可优化、可度量的部分。

注意：用户的主观体感延迟还包含段前的 VAD 切段等待——即"用户说完一句话"到"这句话被切成一段"之间的等待。这部分是采用整段 ASR（非流式）的结构性延迟，**不计入本文 1 秒以内的指标口径**，但可以通过缩短切段窗口来减小（见 7.3）。本设计不引入流式 ASR。

二、架构总览

2.1 架构风格

V2 是 前后端分离 + 后端 GPU 单体 + 大模型容器外挂 的混合形态：

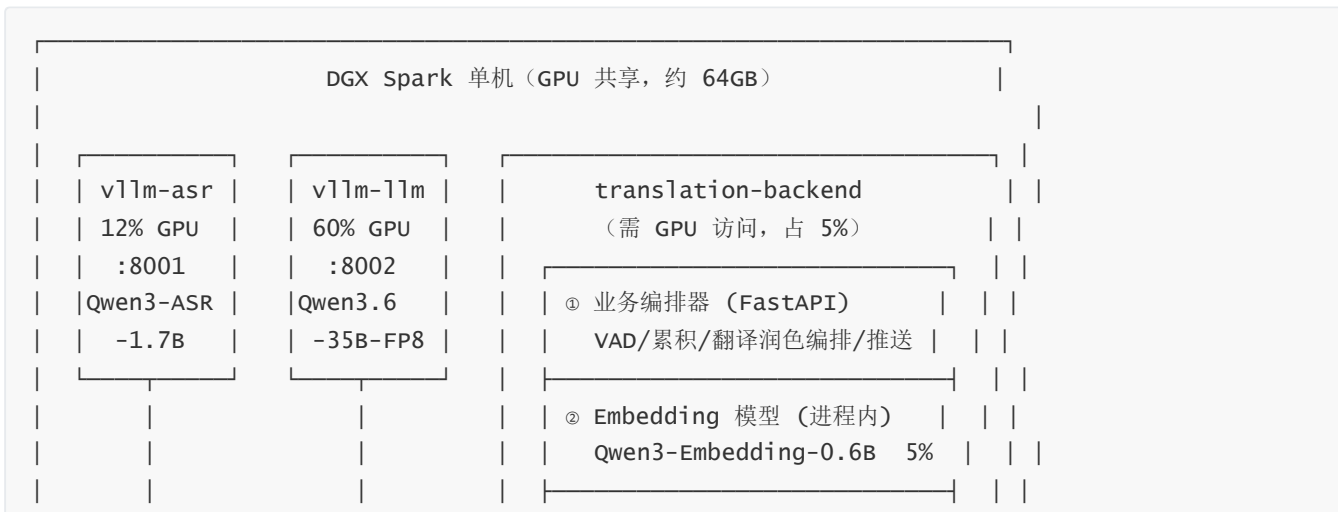
- 前端 = 独立 SPA**：Vue 单页应用，独立构建、独立部署，通过跨域 HTTP/WebSocket 访问后端。
- 后端 = GPU 异步单体**：FastAPI + asyncio 单进程，**首次需要 GPU 访问**（进程内加载 embedding 模型），编排整条业务流水线，进程内持有 FAISS 向量索引。
- 大模型 = vLLM 容器外挂**：ASR 与 LLM 各一个 vLLM 容器，共享 GPU，通过 HTTP 调用。

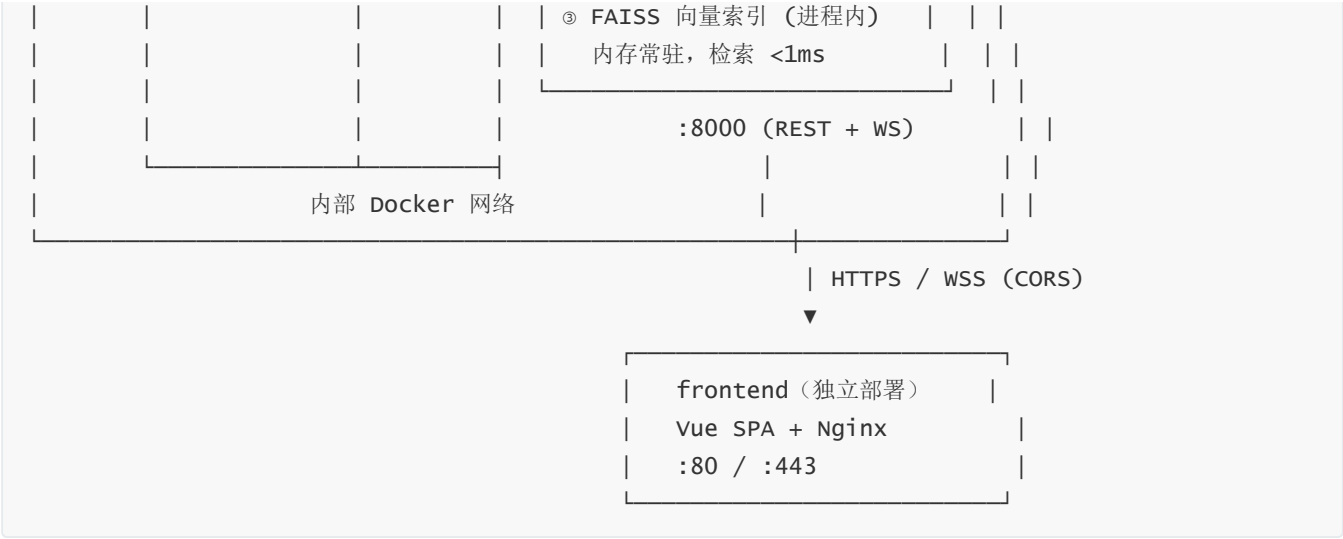
2.2 核心架构原则

- 差异化部署原则**：模型放进程内还是容器外挂，由"模型大小 $\times$ 调用频率 $\times$ HTTP 开销占比"决定。大而重的进程外（容器），小而频的进程内。
- 延迟优先原则**：高频调用必须消除网络跳点；关键路径上的串行环节尽量异步化或条件跳过。
- 零中间件原则**：向量检索内存常驻，不引入独立向量库服务。
- 部署边界假设**：单体应用部署于可信局域网内，安全策略以"内网可信 + 延迟优先"为前提，避免引入增加延迟的强鉴权环节。

三、部署架构

3.1 部署拓扑





3.2 部署单元清单

单元	技术栈	GPU	端口	职责
frontend	Vue SPA + Nginx	无	80/443	音频采集、字幕渲染、导出触发
translation-backend	FastAPI + asyncio + faiss + embedding	5%	8000	业务编排 + 进程内 RAG
vllm-asr	vLLM / Qwen3-ASR-1.7B	12%	8001	语音识别
vllm-llm	vLLM / Qwen3.6-35B-FP8	60%	8002	翻译 + 润色 (共享容器)

3.3 GPU 显存分配

服务	占比	说明
vllm-llm	60%	35B-FP8 权重 + KV cache
vllm-asr	12%	1.7B 全精度
translation-backend (embedding)	5%	Qwen3-Embedding-0.6B 进程内
预留缓冲	23%	KV cache 抖动 + 未来扩展

3.4 容器构成

- 3 容器 (asr / llm / backend) + 1 个独立前端部署
- embedding 模型折叠进后端进程内，前端作为独立单元部署。

四、通信架构

4.1 通信端点矩阵

通道	协议	方向	跨域	数据
/ws/audio	WebSocket binary	前端 → 后端	✓	16kHz/16-bit PCM 帧
/ws/audio	WebSocket text	前端 ↔ 后端	✓	JSON 控制信令 / 实时事件
GET /api/export/{id}	HTTP	前端 → 后端	✓	下载 Markdown 文稿
GET /health	HTTP	任意	—	容器健康检查
backend → vllm-asr	HTTP	后端 → ASR 容器	—	WAV 音频段 → 文本
backend → vllm-llm	HTTP	后端 → LLM 容器	—	翻译/润色 prompt → 译文
backend → embedding	进程内调用	进程内	—	文本 → 向量（无网络）
backend → FAISS	进程内调用	进程内	—	向量 → top-k 术语（无网络）

4.2 前后端分离带来的网络边界

前后端分离把“浏览器 ↔ 后端”从进程内信任关系变为两个独立 origin 间的访问。由于整体部署在可信局域网内，这里采用轻量策略，不引入强鉴权：

- 1. **CORS**：allow_origins 配置为前端所在主机/网段即可，无需逐域名收敛。
- 2. **不强制 Token 鉴权**：内网可信环境下省去接入鉴权，避免额外握手延迟；如后续暴露到更大网络范围再行加固。

五、数据流与流水线

5.1 单向流水线（★ = 目标设计，当前尚未接入）





当前实现：VAD 切段 → 整段 ASR → 语气词过滤/语义累积 → 翻译 → 串行润色 → 推送。第六章的进程内 RAG 为本设计的目标特性，翻译 prompt 尚未注入检索术语。

5.2 双层缓冲

层	位置	切分依据	解决的问题
第一层	AudioBuffer	音频时长 + 静默	ASR 模型输入约束
第二层	语义累积	字符数 + 句末标点	翻译质量的上下文需要

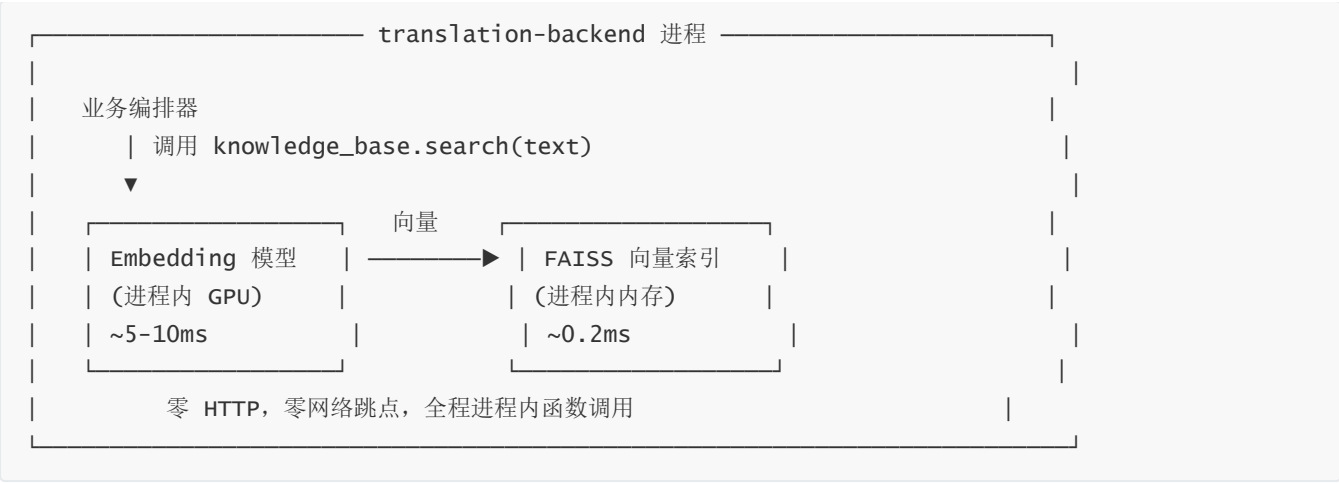
第一层切段窗口当前为 2-5s / 静默 800ms（见 7.3）。

六、RAG 子系统架构（目标设计，尚未接入翻译流程）

6.1 核心决策：全进程内 RAG

若把 embedding 放独立 vLLM 容器、向量化走 HTTP，在 Python 异步 HTTP 栈下，单次小请求的框架开销 + 跨容器往返会引入 **50-100ms 延迟**，且在每段翻译都要付出，无法忽略。

V2 的方案：**Embedding 模型与 FAISS 索引均进程内化**。



6.2 RAG 链路延迟

方案	embedding	检索	总计	评价
embedding 独立容器 HTTP	50-100ms	0.2ms	50-100ms	每段付出，不可忽略
全进程内 (V2 采用)	5-10ms	0.2ms	~5-10ms	可忽略

6.3 向量化与检索策略

- **Embedding 模型**：Qwen3-Embedding-0.6B（进程内加载）
- **向量库**：FAISS（内存 flat index, <1ms）
- **向量化策略**：源词 + 目标词拼接编码，支持跨语言检索
- **注入方式**：检索到的术语作为"假对话"约束注入翻译 prompt
- **降级策略**：RAG 失败返回空术语列表，不阻塞主翻译流程

七、延迟预算与低延迟设计

7.1 端到端延迟预算（段就绪 → 译文）

环节	延迟	性质
WS 上行	<5ms	内网
ASR 转写	200-500ms	跨容器 HTTP（第一瓶颈）
RAG 检索（进程内）	5-10ms	进程内（★目标特性，尚未接入）
LLM 翻译	~100ms	跨容器 HTTP（已优化）
LLM 润色	~100ms	跨容器 HTTP（可移出关键路径）
WS 下行	<5ms	内网
合计	~425-725ms	✅ 满足 1 秒以内指标

关键结论：当 LLM 翻译/润色各压到 100ms 后，LLM 不再是延迟瓶颈。指标口径内的主要环节是 ASR 转写；切段等待在口径之外，但同样影响体感。

7.2 瓶颈一：ASR 转写

ASR 200-500ms 是关键路径上最大的单项。保证 ASR 稳定接近 200ms（音频段越短延迟越低，但需平衡识别准确率），是压低指标口径内延迟的主要手段。

7.3 瓶颈二：VAD 切段等待（结构性延迟）

当前切段策略为 **最短 2s / 最长 5s + 静默 800ms 触发**（见 config.py：asr_buffer_min_ms=2000 / asr_buffer_max_ms=5000 / asr_silence_duration_ms=800）。这意味着 **切段本身就引入约 0.8-5 秒的等待**，发生在 ASR 之前。这部分等待在 1 秒以内指标口径之外，但直接影响用户体验。

优化方向（可选，尚未实施）：

- **缩短切段窗口**：调小 asr_buffer_min_ms / asr_silence_duration_ms，依赖第二层语义累积补偿语义碎片化。代价是 ASR 段变短可能影响识别准确率，需实测权衡。

7.4 瓶颈三：润色串行开销

当前实现为**固定串行**：每段翻译完成后必定调用一次润色（pipeline._translate_pending_source → translator.translate → polisher.polish），润色 ~100ms 串行叠加在关键路径上。

可选优化方向（尚未实施）：

1. **条件跳过**：短句直接出译文，不润色。
2. **异步补更新**：翻译初稿先推前端显示，润色完成后发 update 事件替换。

7.5 现状与优化空间

环节	当前实现	可选优化（尚未实施）
ASR	整段 WAV 接口	保持稳定低位即可
切段等待	2-5s 窗口 / 静默 800ms	缩短切段窗口
润色	固定串行	条件跳过 / 异步补更新
RAG 术语注入	未接入翻译 prompt	落地进程内 RAG（见第六章目标设计）

八、管理控制台（配置中心）

8.1 动机

提示词与运行参数当前分散且需改代码、重启容器才能调整：翻译/润色提示词硬编码在各 LLM 服务模块，推理参数（temperature、max_tokens、上下文窗口）与切段参数固化在 config.py / .env。管理控制台的目标是把这些**运行期可调项收口到一处**，提供受控的查看与修改入口，改完即时生效，无需改代码或重建镜像。

8.2 管理范围

类别	典型项
提示词	翻译/润色 system prompt（按语向区分）
推理参数	temperature、max_tokens、上下文窗口
切段/语义参数	VAD 窗口、静默时长、语义累积阈值
RAG（目标特性）	术语库条目、top-k

不纳入：GPU 显存切分、容器编排等部署期参数。

8.3 架构定位

管理控制台是 前端一个视图（/admin）+ 后端一组管理 API + 一个集中配置存储 的组合。后端引入配置存储作为运行期配置的单一可信源，翻译/润色/切段各服务统一从中取值；修改配置后热加载，对后续翻译生效。内网部署下管理接口不强制鉴权，配置持久化为单个文件，保持零中间件原则，不引入数据库。

本章为目标设计，当前尚未实现。

九、安全与可观测性

9.1 安全（内网部署，延迟优先）

整体部署于可信局域网内，单机单体形态，安全策略从简，避免为安全引入额外延迟：

- **CORS**：allow_origins 配为前端主机/网段，够用即可。
- **不强制接入鉴权**：内网可信，省去 Token 握手；管理接口同样不强制鉴权。
- **模型只读挂载**：宿主机模型目录 :ro 挂载。
- **离线运行**：TRANSFORMERS_OFFLINE=1，模型容器不联网。

前提是部署网络可信。一旦后端需要暴露到办公网之外或公网，应重新引入 Token 鉴权与 Origin 校验。

9.2 可观测性（低延迟目标要求精细埋点）

为支撑 1 秒以内的延迟目标，关键路径必须可观测：

- **分项延迟埋点**：VAD 等待、ASR、RAG、翻译、润色各环节单独计时。
- **结构化日志**：每段翻译输出 trace（session_id + 各环节耗时）。
- **指标暴露**：可选 Prometheus 端点，监控 p50/p95 端到端延迟。

没有分项埋点，就无法定位"为什么这一段比平常慢"——这是延迟 SLA 的必要基础设施。